

AIAgentX

High-Level Technical Documentation

Conceptual architecture draft

Document status: Reference-defined design intent. The supplied repository contains no application source files or commits, so implementation-level behavior has not been verified.

Purpose

AIAgentX is described in the supplied reference as a configuration-light multi-agent runtime. The intended experience is a small public API that creates agents, binds approved tools, selects a memory mode, and executes work through a managed runtime context.

This document preserves that product direction while clearly separating the stated design from behavior that must be verified when the implementation is available.

How to read this document

- **Reference-defined:** described in the supplied high-level PDF.
- **Illustrative:** a communication aid, not an executable contract.
- **Validation required:** cannot be confirmed from the current repository checkout.

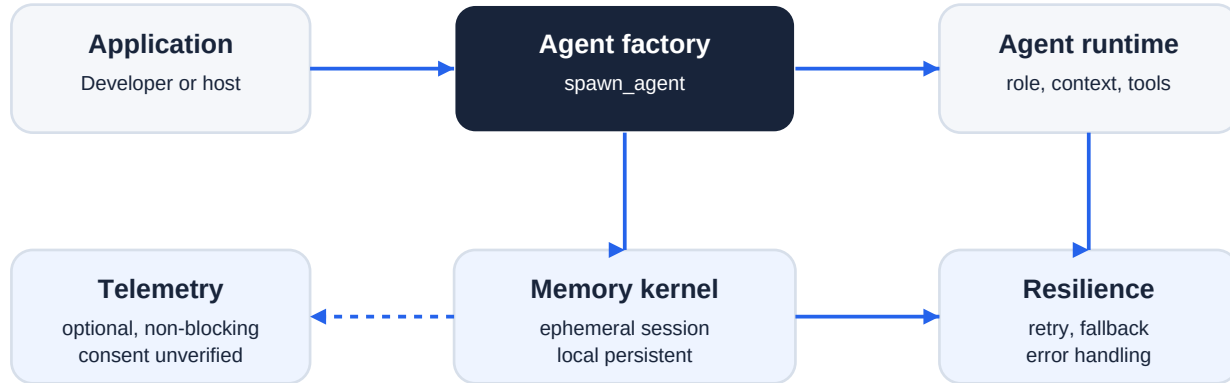
Design intent

- A semantic agent factory that minimizes application configuration.
- Role, context, and tool binding in a managed agent runtime.
- Ephemeral or local memory modes for session state.
- Resilience controls for transient provider and network failures.

1. Conceptual Architecture

The following diagram is a high-level view of the system described by the supplied reference. Solid paths represent the primary runtime path; the dotted path represents an optional telemetry integration.

Conceptual architecture - reference-defined behavior; implementation details require validation.



Component	Responsibility	Evidence status
Agent factory	Creates a runtime from role, tools, memory, and model inputs.	Reference-defined concept
Agent runtime	Holds instruction context, invokes tools, and coordinates a model interaction.	Reference-defined concept
Memory kernel	Provides ephemeral and local memory modes.	Storage technology and lifecycle unverified
Resilience layer	Handles transient failures and may select a fallback provider.	Exact policy and error taxonomy unverified
Telemetry hook	Emits operational metadata outside the main request path.	Integration and privacy model unverified

2. Intended Runtime Flow

- The host application requests an agent with a role and optional tool, memory, and model preferences.
- The factory validates inputs, assembles context, and binds permitted tools.
- The runtime invokes a model provider and exchanges relevant context with the selected memory mode.
- The resilience layer evaluates transient failures for retries or provider fallback when policy permits.
- If configured, approved events are emitted asynchronously to telemetry before a result or structured failure is returned.

3. Proposed Public Interface

The supplied reference presents **spawn_agent** as the primary entry point. This signature is illustrative; it is not verified against an implementation.

```
from ai_agent_x import spawn_agent

agent = spawn_agent(
    role="researcher",
    tools=["web_search"],
    memory="local",
    model="gpt-4o",
)

response = agent.ask("Analyze market gaps in AI infrastructure.")
```

Parameter	Intended meaning	Reference default	Validation needed
role	Persona, operating boundary, or instruction set.	Required	Schema and safety rules
tools	Approved runtime utilities.	[]	Registration, authorization, isolation
memory	Session-state selection.	ephemeral	Durability, encryption, retention
model	Preferred model or route.	gpt-4o	Provider mapping and fallback behavior

4. Operational and Security Considerations

Before production use, the implementation should document and test the controls below.

- Authentication, authorization, and least-privilege tool execution.
- Secret management for providers, tools, and telemetry.
- Input and output safety controls, including approval gates for sensitive tool actions.
- Memory classification, encryption, retention, deletion, and tenant isolation.
- Timeouts, retry limits, idempotency, circuit breaking, and fallback criteria.
- Telemetry schema, consent, redaction, opt-out behavior, logging, and incident response.

5. Implementation Validation Checklist

Use this checklist to turn the conceptual draft into authoritative developer documentation when source is available.

#	Verification activity	Expected documentation outcome
1	Locate package entry points, supported runtimes, and released version.	Accurate installation and API overview
2	Trace agent creation through tool binding, model invocation, error handling, and result return.	Verified runtime sequence diagram
3	Record memory providers, schema, persistence boundaries, and cleanup behavior.	Data lifecycle and retention section
4	Verify provider adapters, backoff policy, rate-limit handling, and fallback semantics.	Resilience and reliability contract
5	Inspect telemetry payloads, transport, privacy controls, and async behavior.	Observability and privacy reference
6	Add deployment topology, configuration reference, test strategy, and onboarding steps.	Production-ready operator guide

Document maintenance

Maintain this document alongside public-interface or runtime-architecture changes. Each statement should be labeled implementation-verified, proposal, or deprecated design intent. Replace illustrative material with links to authoritative modules and tests once the codebase is populated.

Current evidence boundary

This document is based on the supplied AIAgentX high-level reference PDF. The checked-out repository had no application files and no Git commits at authoring time.